Okay, here is the complete design document with the citation markers removed.

ASV Simulator Design Document

1. Overview

This document describes the design of a Python-based simulator for an Autonomous Surface Vehicle (ASV) tasked with navigating a pre-qualification course. The course involves passing through a gate, maneuvering around a marker pole, and returning through the gate. The simulator provides a 2D top-down map view and a simulated 3D first-person camera view. It models an ASV operating on the surface ($Z=0$) using a differential drive (two thrusters). The AI employs a modular Task/Subtask architecture for mission execution and relies on simulated computer vision (OpenCV blob detection) for navigation.

2. Key Program Concepts

- **Task/Subtask Architecture**: The core AI logic is structured hierarchically:
    - **Tasks** (ai.tasks.task_base.Task): Represent high-level mission goals (e.g., GateTask, RatchetTurnTask). Each task manages a sequence of subtasks. Tasks receive the updated Visionobject in their execute method. Tasks might still contain logic based on vision results (e.g., the gate-found check in RatchetTurnTask.execute).
    - **Subtasks** (ai.tasks.subtask_base.Subtask): Represent reusable, atomic actions (e.g., TurnToHeading, DriveStraight, AlignToObjectX). They are executed sequentially by the parent Task. Subtasks receive the updated Vision object in their execute (and on_enter/on_exit) methods. They call the interpretation methods of the Vision object (e.g., vision.is_pole_visible(), vision.get_pole_center_x()) to get the data needed for their actions.

    - **Context Dictionary**: A shared dictionary (Task.context) is used for indirect communication between subtasks within a Task. For instance, an alignment subtask stores the final heading, which a subsequent turning subtask can retrieve to calculate a relative turn.

- **Physics Model** (simulator.py:applyPhysics): Simulates a simplified 2D rigid body model for the ASV.
    - **Differential Drive**: Calculates surge (forward/backward) force and yaw (turning) torque based on independent port and starboard thruster commands.
    - **Drag**: Implements distinct quadratic drag coefficients for surge (surgeDragCoeff) and sway (swayDragCoeff, typically much higher for a boat hull), plus angular drag (angularDragCoeff) resisting rotation.
    - **Inertia**: Includes mass (subMass) and rotational inertia (subInertia) affecting linear and angular acceleration.
    - **Surface Constraint**: The boat's Z-position and pitch angle are locked to zero.
- **Simulated Vision (Vision Class & ai/vision.py)**:
    - The core vision processing is encapsulated within the Vision class defined in data_structures.py.
    - **Initialization**: An instance of the Vision class is created once within the Submarine AI's constructor. It receives an image_provider function (typically a lambda accessing the latest SensorSuite.camera_image) and configuration parameters like minimum pixel thresholds.
    - **Processing Trigger**: At the beginning of each Submarine.update cycle, the vision.update() method is called. This method fetches the current camera image via the image_provider, runs blob detection (find_blobs_hsv from ai/vision.py) for relevant color ranges (e.g., green pole, red gate poles), stores the raw blob lists internally, and clears any cached interpretations from the previous frame.
    - **Interpretation**: The Vision class provides methods (e.g., is_pole_visible(), get_gate_center_x(), get_pole_apparent_height()) that interpret the internally stored blob lists to answer specific questions about detected objects. These methods often cache their results within a single frame for efficiency.

    - **Blob Detection (ai/vision.py)**: The find_blobs_hsv function remains the low-level tool using OpenCV (cv2) to find blobs based on HSV color ranges and minimum size.

- **Configuration** (config.py): Centralizes constants, tuning parameters (PID gains, drag coefficients, AI thresholds), color definitions (RGB and HSV), and course layout dimensions.

3. Program Structure

- **main.py**:
    - The main entry point for the application.
    - Defines the mission plan as an ordered list of Task objects. Sets task-specific parameters like search_direction if needed.
    - Instantiates the Submarine (AI controller) and the SubmarineSimulator.
    - Starts the simulation loop (sim.run()).
- **simulator.py**:
    - Contains the SubmarineSimulator class.
    - Initializes Pygame and creates the display window.
    - Loads assets (e.g., camera background image).
    - Manages the main simulation loop (run method):
        - Handles user input (handleInput).
        - Updates the AI (submarineAI.update).
        - Applies physics based on thruster commands (applyPhysics).
        - Generates the 3D camera view (generateCameraView, project3D).
        - Renders the 2D map and UI elements (render, _renderUi). Includes debug drawing based on results from the Vision object.
    - Defines physics parameters (mass, inertia, drag coefficients).

    - Creates the course elements (resetSimulation).

- **config.py**:
    - Defines color constants (RGB tuples).
    - Defines HSV color range tuples/lists for vision detection.
    - Contains dataclasses (SimulationConfig, PrequalConfig) holding world dimensions, camera FOV, boat dimensions, course layout, physics constants, and AI tuning gains.
- **world.py**:
    - Defines dataclasses representing physical objects (PrequalGate, PrequalMarker).
    - Defines SubmarinePhysicsState dataclass to hold the boat's position, orientation, and velocities.
- **data_structures.py**:
    - Defines simple dataclasses for data transfer: ThrusterCommands, SensorSuite, MPU6050Readings.
    - Defines the Vision class, which encapsulates blob detection and interpretation. It takes an image provider during initialization and has an update() method to process the latest image.
- **utils.py**: Contains helper functions (e.g., angle_diff).
- **ai/**:
    - **submarine.py**:
        - Contains the main Submarine AI class.
        - __init__: Stores the mission plan, initializes AI gains. Creates and stores a single Vision object instance, providing it with a way to access the camera image from the latest SensorSuite.
        - update: Calls self.vision.update() at the start of each cycle. Executes the current Task, passing the self.vision object. Handles task transitions.
        - reset: Resets the mission state.

        - Low-level control methods (called by Tasks/Subtasks): _mix_and_normalize_commands, get_heading_commands, _get_pid_hover_commands, _get_damping_commands, get_search_commands, get_go_to_visual_target_commands.

    - **vision.py**: Contains find_blobs_hsv using OpenCV for color segmentation and contour finding.

    - **tasks/**:

        - **task_base.py**: Defines Task base class (manages subtask list, current_subtask_index, context) and TaskStatus enum. Its execute method runs the current subtask, passing the Vision object, handling subtask completion/failure, context passing, and search spin logic. It no longer calls process_vision.

        - **subtask_base.py**: Defines Subtask base class (defines on_enter, execute, on_exit interface) and SubtaskStatus enum. Method signatures expect a Vision object.

        - **common_subtasks.py**: Implements reusable Subtasks (e.g., TurnToHeading, DriveStraight, Stabilize, WaitForTargetVisible, AlignToObjectX, DriveUntilTargetLost, DriveUntilTargetLostRight, SpinUntilTargetReappearsLeft, ApproachAndCenterObject). Vision-based subtasks now use methods of the Vision object.

        - **gate_task.py**: Task for gate navigation. Defines a subtask sequence using common subtasks. Does not contain a process_vision method.

        - **ratchet_turn_task.py**: Alternative task for marker navigation using align-surge-yaw loop. Defines a subtask sequence and handles loop logic and gate detection within its executemethod. Does not contain a process_vision method.

        - **marker_turn_task.py** (Alternative, requires update): Task for marker navigation using dynamic orbit. Includes custom inline subtasks (_StoreHeadingSubtask, _DynamicOrbitSubtask). *Note: This file was not fully updated to use the Vision class in the latest refactor*.

        - **stabilize_task.py**: Task wrapper around PID hover control, using the Vision object type hint.

4. Transitioning to Hardware

Moving the AI from this simulator to a physical ASV involves replacing simulated components with real-world interfaces and extensive tuning:

1. **Hardware Platform**:
    - Compute: Microcontroller or SBC (Raspberry Pi, Jetson Nano, etc.).
    - Sensors: Camera (USB/CSI), IMU (MPU6050, BNO055) for heading/rates (requires fusion), optional GPS. No depth sensor needed.
    - Actuators: Two Brushless DC Motors/propellers (e.g., T200), ESCs.
    - Power: Batteries, etc..
    - Communication: Optional Wi-Fi/Telemetry.
2. **Software Adaptation**:
    - Replace SensorSuite population: Read from real sensor drivers/libraries. Implement sensor fusion for heading.
    - Real Vision: The find_blobs_hsv function works on real camera frames. HSV ranges in config.pyneed significant retuning. The Vision class structure remains valid.
    - Replace ThrusterCommands Output: Map -1 to 1 thrust values to ESC-compatible signals (e.g., PWM).
    - Remove applyPhysics: The real world handles physics.
3. **Tuning**:
    - PID Gains: All gains (YAW_D_GAIN, HOVER_XY_P/I/D, etc.) need extensive retuning on the actual boat. Start low.
    - Vision Thresholds: HSV ranges, MIN_PIXELS_FOR_DETECTION (used by Vision class) need tuning.
    - Subtask Parameters: Durations, surge powers, completion tolerances need adjustment.
4. **Real-World Considerations**:
    - Latency: Account for sensor/actuator delays.

- Noise: Sensor filtering may be needed.
- Calibration: IMU, Camera (HSV), ESCs need calibration.
- Power Management.
- Environmental Factors: Wind, waves, currents may require more robust control.

The core Task/Subtask logic and the Vision class structure should be largely transferable, but interfacing and tuning require significant effort.

5. Task and Subtask Details

5.1 GateTask - Navigating the Gate

The GateTask finds the gate (red poles), aligns, and drives through.

1. **WaitForTargetVisible(target_type='gate')**:
   - **Goal:** Find the gate visually.
   - **Action:** Runs until vision.is_gate_visible() returns True.
   - **Control:** Spins if gate not visible, damps when found.
2. **AlignToObjectX(target_x_fraction=0.5, ...)**:
   - **Goal:** Point at the gate center.
   - **Action:** Uses vision.get_gate_center_x() and commands yaw to bring this to the screen center (0.5) using PD control.
   - **Control:** Commands yaw only. Completes when centered and stable. Stores heading in context.
3. **Stabilize(duration=1.0, ...)**:
   - **Goal:** Stop residual motion.

- **Action:** Uses PID position control (_get_pid_hover_commands) to hold position/heading.
- **Control:** Runs for 1.0s, completes if speed is low.

4. **DriveUntilTargetLost(surge_power=0.6, target_type='gate')**:
   - **Goal:** Drive straight through the gate.
   - **Action:** Retrieves heading from context, commands forward surge while maintaining heading.
   - **Control:** Continuously checks vision.is_gate_visible(). Returns RUNNING while visible, COMPLETED once lost (after being seen once).
5. **DriveStraight(duration=4.0, ...)**:
   - **Goal:** Ensure boat fully clears the gate.
   - **Action:** Continues driving straight.
   - **Control:** Runs for 4.0s, then completes.
6. **Stabilize(duration=1.0)**:
   - **Goal:** Stop after clearing the gate.
   - **Action:** Uses PID control to halt motion.
   - **Control:** Runs for 1.0s, checks speed before completing.

Completion allows the mission to proceed.

5.2 Marker Turn Task - Maneuvering Around the Marker (Dynamic Orbit - Requires Update)

*(Note: This task implementation (marker_turn_task.py) was **not fully updated** to use the Vision class in the latest refactor and would require modification before use)*.

The original MarkerTurnTask finds the green pole, orbits, and turns back. Its intended subtask sequence was:

1. WaitForTargetVisible(target_type='pole'): Find green pole. Spin if needed.

2. AlignToObjectX: Align to an offset (e.g., 60%). Store heading.
3. _StoreHeadingSubtask: (Redundant now) Capture heading.
4. _DynamicOrbitSubtask: (Internal subtask) Command constant surge. Use dynamic yaw target moving from start% to end% based on heading change. Check completion angle. Handle target loss with local search. Store final_turn_target.
5. TurnToHeading(relative_degrees=-180.0, ...): Turn back towards the gate using the final_turn_target or calculating from initial_heading.

If the orbit subtask failed, the task's execute override would reset and restart the sequence.

5.3 RatchetTurnTask - Alternative Marker Navigation (Ratchet Method)

As an alternative to continuous orbiting, the RatchetTurnTask navigates around the green marker pole using a discrete, step-by-step ("ratchet") approach until the return gate becomes visible. It uses the following sequence of subtasks, with looping managed by the RatchetTurnTask.execute method:

1. **WaitForTargetVisible(target_type='pole')**:
    - **Goal:** Find the green marker pole visually.
    - **Action:** Runs until vision.is_pole_visible() returns True.
    - **Control:** Spins if pole not visible, damps when found.
2. **AlignToObjectX(target_x_fraction=initial_target_fraction, ...)**:
    - **Goal:** Point towards the pole, offset to the initial side (e.g., 60%).
    - **Action:** Uses vision.get_pole_center_x() and commands yaw to bring it to initial_target_fraction.
    - **Control:** Commands yaw only. Completes when centered and stable. Stores heading.
3. **DriveUntilTargetLostRight(surge_power=..., disappear_threshold_fraction=...)**: (Start of Loop)

- **Goal:** Drive straight past the pole until it leaves view on the right.
- **Action:** Retrieves heading, commands constant surge, maintains heading. Monitors pole visibility and last center_x.
- **Control:** Completes when pole becomes not visible *and* was last seen beyond the disappear_threshold_fraction.

4. **SpinUntilTargetReappearsLeft(target_fraction=subsequent_target_fraction, yaw_power=...)**:
    - **Goal:** Turn left until the pole comes back into view, past the *next* alignment target.
    - **Action:** Commands constant yaw_power. Monitors pole visibility and center_x.
    - **Control:** Completes when is_pole_visible() is True *and* pole_center_x is *less than*subsequent_target_fraction.
5. **AlignToObjectX(target_x_fraction=subsequent_target_fraction, ...)**:
    - **Goal:** Re-align to the pole using the wider subsequent_target_fraction.
    - **Action:** Uses vision.get_pole_center_x() and commands yaw to bring it to subsequent_target_fraction.
    - **Control:** Commands yaw only. Completes when centered and stable. Stores heading.

After subtask 5 completes, RatchetTurnTask.execute resets current_subtask_index to the loop start (index 2) and returns RUNNING.

Completion Check: After each subtask (except index 0), RatchetTurnTask.execute checks vision.is_gate_visible(). If True, it returns COMPLETED.